Auditory attention
API and UML reference

9 September 2026 | audio branch | source snapshot 04ce6af86c9736ac930688ea83ac855ee72d0202

A source-derived reference for the implemented auditory module and its integration scripts. Like the previous streaming/Riemann guide, this document starts with call recipes, then covers contracts, subsystem UML, classes and module functions.

Scope: production classes and top-level functions in src/nova2026/auditory and scripts/auditory. The existing Pipeline API is summarized. Tests and third-party APIs are excluded from the signature inventory. Exact signatures/defaults are extracted from the current Python source; prose states actual behavior and remaining caller responsibilities.

Read first	Where to go
Run it	Recipes: synthetic demo, training, replay and playback
Understand data	Array shapes, time domains and model contracts
Understand ownership	Subsystem UML and relationship register
Call a component	One class per API page; functions grouped by module
Verify independently	Separate NOVA2026_Auditory_Test_Agent_Brief.txt

Status: recorded-data processing, model training, replay rendering and controller software exist. LiveAuditoryAdapter is an integration seam, not a completed synchronized hardware application. Synthetic checks do not establish human decoding accuracy, reliable attention switching or hearing benefit.

UML notation: + is an exposed operation; compartments separate class name, selected state and operations. Solid diamonds in the editable Mermaid source mean ownership; dashed arrows mean uses/creates; hollow triangles mean inheritance. Diagram method lists abbreviate parameters; the API pages contain the exact signatures. State fields are not invitations to mutate fitted models.

Call recipes | train and render

Run these commands from C:/coding/git/NOVA2026 using the existing project environment. Output folders contain generated data, not source code.

```
.venv/Scripts/python.exe -B -m scripts.auditory.demo
.venv/Scripts/python.exe -B -m scripts.auditory.train --train data/train.npz --validation
    data/val.npz --model models/auditory.npz --history 5
.venv/Scripts/python.exe -B -m scripts.auditory.replay --trial data/test.npz --model
    models/auditory.npz --out output/audio_test
```

demo creates synthetic train/validation/test NPZs, a model, mixed.wav and report.json. train creates the model plus a .validation.json report. replay writes mixed.wav, estimates.json and metrics.json. CLI data paths are illustrative unless created by demo.

```
from nova2026.auditory.data import load_trial
from scripts.auditory.train import train
from scripts.auditory.replay import replay

training = [load_trial("data/train.npz")]
validation = [load_trial("data/val.npz")]
model, report = train(training, validation, history=5.0)
model.save("models/auditory.npz")
trial = load_trial("data/test.npz")
audio, estimates, metrics = replay(trial, model)
```

Group repeated excerpts correctly before splitting. The train function checks group and trial overlap; the replay function itself does not. Held-out checks are applied by replay/evaluate CLIs. Enforce subject-disjoint partitions separately when claiming generalization to new participants.

Call recipes | playback and pipeline

```
from nova2026.auditory.data import load_trial
from nova2026.auditory.decoder import RidgeDecoder
from scripts.auditory.streamer import AuditoryReplayStreamer

trial = load_trial("data/test.npz")
model = RidgeDecoder.load("models/auditory.npz")
streamer = AuditoryReplayStreamer(trial, model)
streamer.initialize()
streamer.stream()
```

Optional playback uses sounddevice. Install the declared audio extra if needed: pip install -e ".[audio]". Playback is not started by generating these documents. streamer.stream blocks and owns teardown; streamer.stop signals cancellation from another thread. Call initialize before another run. There is no close method on AuditoryReplayStreamer.

```
from nova2026.auditory.pipeline import AuditoryPipeline

# clock must return the same time domain used by the window timestamps.
pipeline = AuditoryPipeline(model, clock)
estimate, original = pipeline.rundown(aligned_window)

# Alternative inherited step API: there is one registered predict tube.
pipeline.feed(aligned_window)
estimate, original = pipeline.step()
```

Inherited Pipeline calls: add_tube(tube), rundown(data), feed(data), step(steps=1), spit(). A tube returns (result, next_data). step clears the retained load after the final tube. spit returns and clears an unfinished load, not an already-completed estimate.

For live integration, construct LiveAuditoryAdapter(pipeline, envelopes, handoff) and pass adapter.rundown to the existing Streamer with pipeline_on_invalid=True. The caller must supply audio acquisition, verified clock mapping, and bounded transport into the processing-thread-owned EnvelopeBuffer. This is an integration recipe, not a ready-to-run hardware example.

Contracts | arrays and labels

Value	Shape / meaning
Trial EEG	(N,C), numerical uV; named channels in declared order
Trial audio	(M,2), full-band candidate waveforms; mixer expects [-1,1]
Trial labels	(N,), -1 unknown, 0 candidate A, 1 candidate B
Aligned EEG	(N,C), causally processed common-rate samples
Aligned envelopes	(N,2), same grid as the aligned EEG
Timestamps	(N,), increasing seconds; one consistent clock domain
Decoder output	(2,) Pearson correlations, not probabilities
Mixer output	(M,), mono float32 with fixed headroom
Controller choice	0, 1 or None; evaluation encodes neutral as -1

AuditoryTrial contains labels; AuditoryWindow does not. Candidate identity is independent of attention. KU Leuven candidates are ordered by filename, then the known attended track is mapped to the corresponding label. The audible output is not filtered to 1-9 Hz; only EEG and speech-envelope features use the slow band.

Trial NPZ keys: eeg, timestamps, audio, labels, metadata. Metadata includes audio_rate, subject, trial_id, channel_names, reference, upstream_processing and group. save_trial writes arrays and JSON. load_trial does not execute pickled data.

The current interchange assumes audio sample zero coincides with trial.timestamps[0]. Confirm or correct onset cropping and units at import. AASD requires two candidate signals: two spatial channels of an already mixed recording are not necessarily two separate voices.

Contracts | timing, normalization and failures

Signal timestamps identify nominal sample time. available_at records conservative availability after filtering/resampling/batching. emitted_at records inference completion. evidence_end refers to the final stimulus time supported by the decoder lags, so it precedes the final EEG timestamp by the lag span.

Default feature rate is 64 Hz; lag_seconds=0.4 rounds to L=26 samples. A window with N EEG rows yields N-L reconstructed envelope rows. Row t uses EEG[t] through EEG[t+L]. Latest incomplete rows are excluded. Neural lag, filter phase delay, transport delay and sound-card delay are distinct quantities.

EEG uses the existing RunProcessor and stateful filters/resampler. Audio uses rectification, eighth-order 20 Hz low-pass, third-order feature band-pass and continuous-grid interpolation. Both paths retain state across chunks. The envelope extractor returns no timestamps itself; runner.py assigns them and records availability.

Model metadata contains feature configuration, EEG contract, normalization and training provenance. validate checks the saved contract, but metadata checks cannot establish that a source really used the claimed reference or units. Direct score calls do not gate window.valid; use AuditoryPipeline for that behavior.

Invalid aligned windows bypass model scoring and return invalid estimates. The controller rejects invalid/nonfinite evidence and expires automatic decisions by evidence age. Manual mode explicitly overrides automatic expiry. A ReplayFailure means the EEG run stopped; offline replay records processing_failure and can finish audio rendering, while playback facade failures propagate to the caller.

Default source batching can make valid-looking evidence too old to act on. Coverage must be reported alongside accuracy. Do not silently increase max_age to make a demonstration look active. The baseline named hysteresis retains an accepted choice without the quality controller's expiry/neutral behavior.

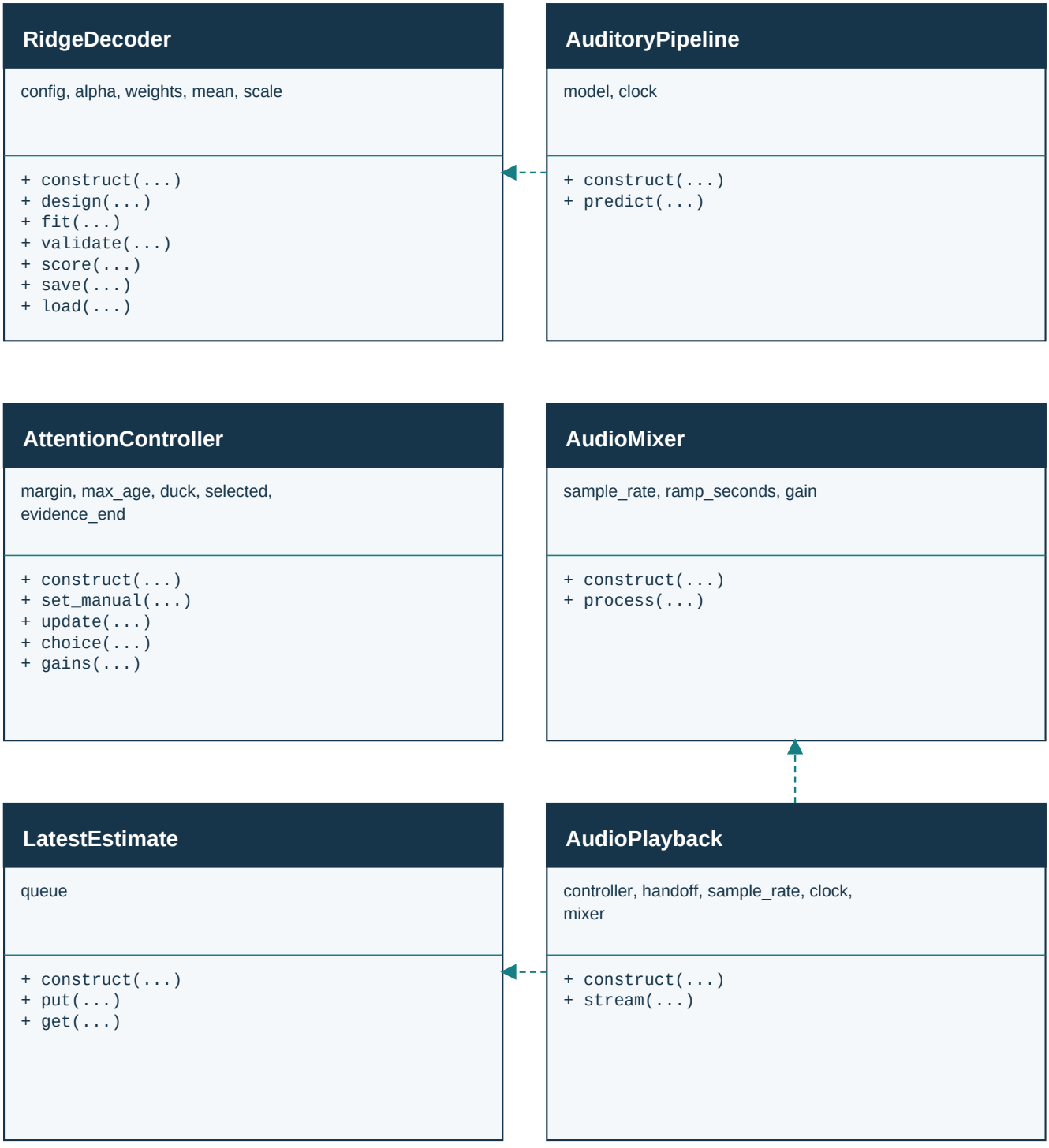
UML 1 | Data and continuous processing

Data and continuous processing

Dashed arrows in this view indicate selected dependencies. Consult the relationship register for complete cross-subsystem links and ownership.

UML 2 | Decoding and audio control

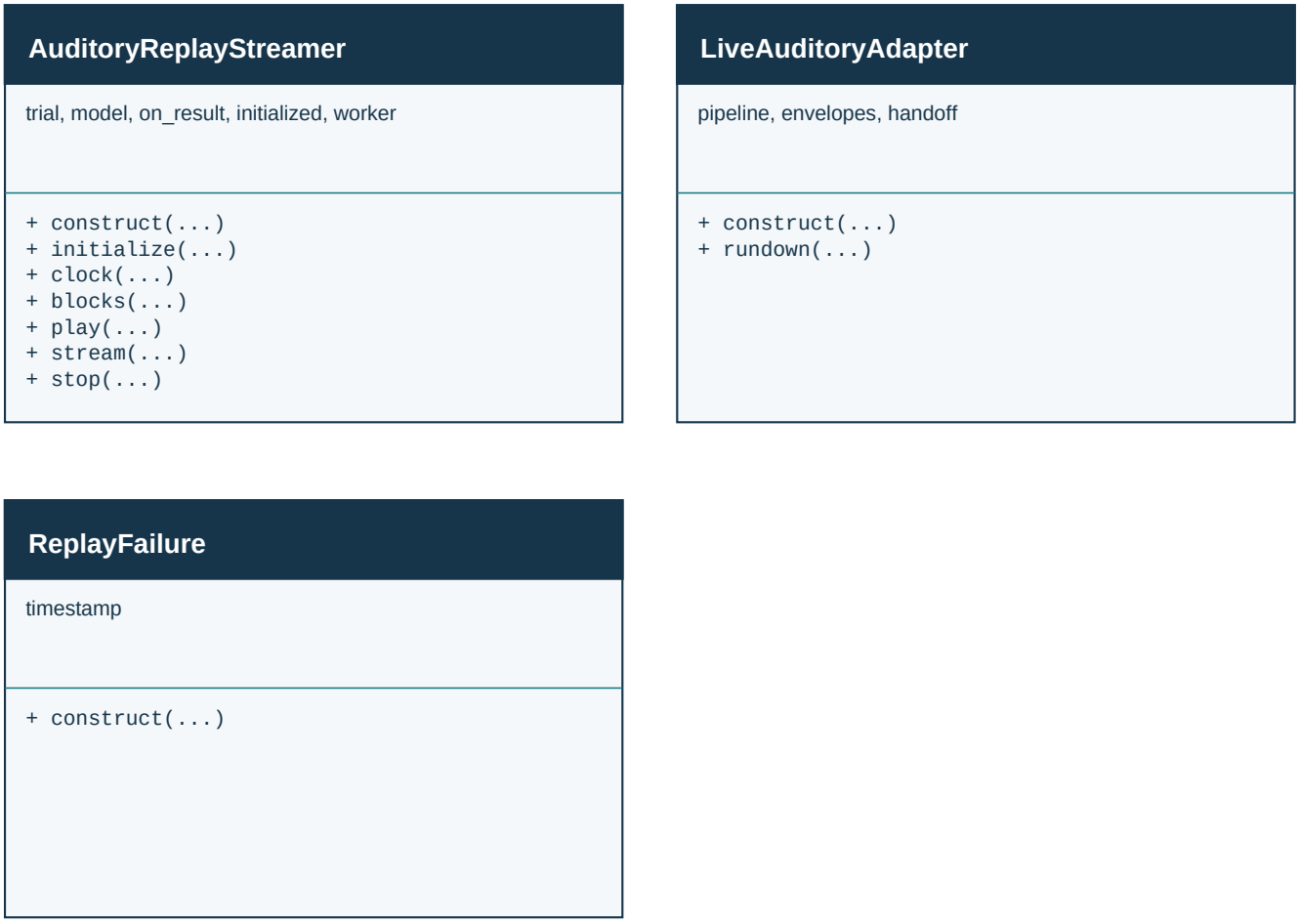
Decoding and audio control



Dashed arrows in this view indicate selected dependencies. Consult the relationship register for complete cross-subsystem links and ownership.

UML 3 | Replay and live integration

Replay and live integration



Dashed arrows in this view indicate selected dependencies. Consult the relationship register for complete cross-subsystem links and ownership.

UML | relationship register

From / to	Relationship
AuditoryPipeline -> Pipeline	inherits
AuditoryPipeline -> RidgeDecoder	uses supplied model
AuditoryPipeline -> AttentionEstimate	creates
EnvelopeExtractor -> AuditoryConfig	uses supplied config
EnvelopeBuffer -> AuditoryWindow	creates in align
AuditoryReplayStreamer -> LatestEstimate	owns
AuditoryReplayStreamer -> AttentionController	owns
AuditoryReplayStreamer -> AudioPlayback	owns
AudioPlayback -> AudioMixer	owns
AudioPlayback -> LatestEstimate	consumes supplied handoff
AudioPlayback -> AttentionController	uses supplied controller
LiveAuditoryAdapter -> EnvelopeBuffer	uses supplied history
LiveAuditoryAdapter -> AuditoryPipeline	uses supplied pipeline
LiveAuditoryAdapter -> LatestEstimate	publishes estimate
ReplayFailure -> RuntimeError	inherits

replay_windows is a function, not a class. It constructs RunProcessor, EnvelopeExtractor and EnvelopeBuffer, then creates aligned windows. train creates RidgeDecoder candidates. replay creates pipeline/controller/mixer components. These function-local objects are intentionally not modeled as new framework classes.

Class API | EnvelopeBuffer

```
from nova2026.auditory.alignment import EnvelopeBuffer
```

Bounded candidate-envelope history. The processing thread owns this object. Alignment interpolates within available support and never extrapolates beyond it.

Source: src/nova2026/auditory/alignment.py : 8

```
EnvelopeBuffer(sample_rate, seconds: float=60)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
reset()
```

Clears envelope values, timestamps and availability arrays.

```
feed(values, timestamps, available_at)
```

Appends (K,2) values on a continuous sample_rate grid. Checks finite data, continuity and available_at >= final signal timestamp. Keeps only capacity samples.

```
align(window)
```

Accepts an existing EEGWindow. Returns AuditoryWindow with aligned candidate envelopes, inherited EEG validity/contract/segment, and combined availability. Missing support adds audio_unavailable.

Class API | AudioMixer

```
from nova2026.auditory.audio import AudioMixer
```

Mix two normalized full-band audio columns with per-sample exponential gain ramps and fixed 0.49 headroom. This is gain control, not speech separation.

Source: src/nova2026/auditory/audio.py : 26

```
AudioMixer(sample_rate, ramp_seconds=0.4)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
process(samples, target)
```

Accepts normalized (M,2) samples and gains (2,) in [0,1]. Returns mono float32 (M,). Rejects nonfinite/out-of-range input and updates the current gain at the final sample.

Class API | LatestEstimate

```
from nova2026.auditory.audio import LatestEstimate
```

Single-producer/single-consumer handoff with capacity one. A new pending estimate replaces an older pending estimate; no backlog of decisions is retained.

Source: src/nova2026/auditory/audio.py : 54

```
LatestEstimate()
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
put(estimate)
```

Enqueues an estimate without waiting; replaces the pending item if full. Intended for one producer only.

```
get()
```

Returns the pending estimate or None, without blocking.

Class API | AudioPlayback

```
from nova2026.auditory.audio import AudioPlayback
```

Blocking output loop intended for the independent audio worker. The loop checks evidence expiry each block even when no new estimate arrives.

Source: src/nova2026/auditory/audio.py : 74

```
AudioPlayback(controller, handoff, sample_rate, clock)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
stream(blocks, stop_event)
```

Accepts an iterable of normalized (M,2) blocks and a threading.Event. Opens optional sounddevice output, drains the handoff, checks controller gains, writes audio and counts underflows. Returns None.

Class API | AuditoryConfig

```
from nova2026.auditory.config import AuditoryConfig
```

Feature settings only. Construct before preprocessing and fitting; use a fresh model whenever the feature definition changes.

Source: src/nova2026/auditory/config.py : 6

```
AuditoryConfig(sample_rate=64, band=(1.0, 9.0), lag_seconds=0.4)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
to_dict()
```

Returns JSON-compatible feature metadata, including the envelope method identifier. lag_samples is round(lag_seconds * sample_rate).

Class API | AttentionController

```
from nova2026.auditory.controller import AttentionController
```

Own this object on the audio thread. Gap threshold selects A or B; weak evidence selects neutral. No probabilistic confidence calibration or state model is implemented.

Source: src/nova2026/auditory/controller.py : 8

```
AttentionController(margin=0.03, max_age=3.0, attenuation_db=6.0)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
set_manual(enabled, candidate=None)
```

enabled=True selects an explicit candidate (0/1) or neutral (None) and overrides automatic expiry. Invoke on the controller-owning thread. enabled=False resumes automatic choice.

```
update(estimate, now)
```

Consumes one estimate at now; returns None. Rejects invalid/nonfinite scores and stale/future timestamps; ignores older valid evidence. Gap below margin selects neutral; otherwise selects the higher score.

```
choice(now)
```

Returns 0, 1, or None. Automatic evidence expires when now - evidence_end > max_age. now must be a finite monotonic value in the same clock domain.

```
gains(now)
```

Returns two target gains. Neutral is [1,1]; selected candidate stays at 1 and the other becomes 10**(-attenuation_db/20). Does not mutate the mixer ramp.

Class API | AuditoryTrial

```
from nova2026.auditory.data import AuditoryTrial
```

Dataset container for continuous EEG, two stable audio candidates, and labels used only by training/evaluation. Numeric EEG is in uV. Audio sample zero is assumed aligned to the first EEG timestamp.

Source: src/nova2026/auditory/data.py : 9

```
AuditoryTrial(eeg, timestamps, audio, audio_rate, labels, subject, trial_id, channel_names,
    reference, upstream_processing, group=None)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
sample_rate [property]
```

Read-only property: returns reciprocal median EEG timestamp interval. This does not certify that the entire recording is uniformly sampled.

Class API | AuditoryWindow

```
from nova2026.auditory.data import AuditoryWindow
```

Aligned input without labels. This container stores arrays and provenance; it does not by itself validate their physical correctness.

Source: src/nova2026/auditory/data.py : 55

```
AuditoryWindow(eeg, envelopes, timestamps, available_at, valid=True, reasons=(),
    contract=None, segment=0)
```

eeg: (N,C); envelopes: (N,2); timestamps: (N,). available_at is the conservative source/virtual time when the data became available. segment identifies continuity after recovery.

Class API | AttentionEstimate

```
from nova2026.auditory.data import AttentionEstimate
```

Two correlation scores and timing metadata. Scores are not probabilities. The container accepts values that the controller may subsequently reject.

Source: src/nova2026/auditory/data.py : 79

```
AttentionEstimate(scores, evidence_end, emitted_at, valid=True, reasons=())
```

scores: (2,) or None; evidence_end and emitted_at: seconds in one clock domain. valid and reasons describe usable evidence, not successful classification.

```
to_dict()
```

Returns scores as a list or None, timing, valid and rejection reasons. This is the decision-log representation.

Class API | RidgeDecoder

```
from nova2026.auditory.decoder import RidgeDecoder
```

Train one backward stimulus-reconstruction model. Frozen EEG mean and scale are applied at inference. Each target time uses EEG from that time through the configured positive lag.

Source: src/nova2026/auditory/decoder.py : 10

```
RidgeDecoder(config=None, alpha=100.0)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
design(eeg)
```

Requires fitted mean/scale. Returns (N-L, C*(L+1)) features for L lag samples. Row t concatenates normalized EEG[t], ..., EEG[t+L]; no wraparound or trailing zero padding.

```
fit(examples, training_info=None)
```

Accepts iterable of (AuditoryWindow, per-sample labels). Uses valid windows; labels -1 are excluded from target fitting. Fits EEG normalization and ridge weights. Returns self. The caller must enforce data splits.

```
validate(window)
```

Checks exact saved EEG contract equality, common-grid timestamps, array dimensions and finite values. This is not a guarantee that every claimed metadata field is true.

```
score(window)
```

Accepts an aligned window; returns two Pearson correlation scores. Calls validate, reconstructs N-L target samples and compares both candidates. Zero reconstruction yields zero scores. Direct callers must gate window.valid; AuditoryPipeline does so.

```
save(path)
```

Writes weights, EEG mean/scale and metadata to compressed NPZ. Requires a fitted model; parent-directory creation belongs to the caller or training CLI.

```
classmethod load(path)
```

Class method returning a model. Uses allow_pickle=False; checks version, envelope identifier, array dimensions, finite values and positive scales. Performs no fitting.

Class API | EnvelopeExtractor

```
from nova2026.auditory.envelopes import EnvelopeExtractor
```

Own one extractor per continuous pair of speech streams. Rectification, causal low-pass and band-pass filters precede interpolation onto the common grid. No EEG is processed here.

Source: src/nova2026/auditory/envelopes.py : 9

```
EnvelopeExtractor(audio_rate, config)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
reset()
```

Clears filter state, input/output counters and the preceding sample. Reset when beginning a new continuous audio segment.

```
feed(samples)
```

Accepts finite (M,2) audio. Returns (K,2) envelopes; K may be zero. The extractor returns values only; the caller assigns nominal timestamps and records availability.

Class API | AuditoryPipeline

```
from nova2026.auditory.pipeline import AuditoryPipeline
```

A subclass of the existing Pipeline. It registers one predict tube and returns an estimate together with the original AuditoryWindow.

Source: src/nova2026/auditory/pipeline.py : 8

```
AuditoryPipeline(model, clock)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
predict(window)
```

Invalid windows bypass model.score and return scores=None. evidence_end is timestamps[N-L-1]; emitted_at comes from the supplied clock callable. Returns (AttentionEstimate, original_window).

Class API | ReplayFailure

```
from scripts.auditory.runner import ReplayFailure
```

RuntimeError carrying the virtual-clock timestamp at which the EEG processor stopped. It distinguishes a failed run from a normal recording tail.

Source: scripts/auditory/runner.py : 11

```
ReplayFailure(timestamp, message)
```

Stores timestamp and initializes RuntimeError with the failure message.

Class API | LiveAuditoryAdapter

```
from scripts.auditory.runner import LiveAuditoryAdapter
```

Integration helper for the existing Streamer. It does not acquire or synchronize audio. The caller must drain a bounded timestamped audio-feature queue on the processing thread.

Source: scripts/auditory/runner.py : 115

```
LiveAuditoryAdapter(pipeline, envelopes, handoff)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
rundown(window)
```

Aligns an EEGWindow, calls AuditoryPipeline.rundown, forwards the estimate through LatestEstimate and returns the unchanged pipeline result. Enable pipeline_on_invalid in the existing Streamer.

Class API | AuditoryReplayStreamer

```
from scripts.auditory.streamer import AuditoryReplayStreamer
```

Recorded-EEG playback facade. The caller computes envelopes and inference; a separate worker plays audio. Its monotonic clock is not a measured sound-card DAC clock.

Source: scripts/auditory/streamer.py : 13

```
AuditoryReplayStreamer(trial, model, on_result=None)
```

Constructs the component with the parameters above and initializes its owned state. Follow the array and ownership contracts described in this guide.

```
initialize()
```

Creates a fresh handoff/controller, clears stop/error state, and prepares one run. Raises if the previous audio worker is still alive. Does not connect an EEG headset.

```
clock()
```

Internal-use helper mapping monotonic elapsed replay time to the recording start. Available after stream establishes started_at.

```
blocks()
```

Internal-use generator yielding approximately 32 ms audio blocks, including a final partial block.

```
play()
```

Worker target. Runs AudioPlayback, stores exceptions for the caller, and sets the stop event on exit.

```
stream()
```

Requires initialize. Starts an audio worker; runs causal recorded replay on the caller, waits until virtual availability, emits estimates and calls on_result((estimate, window)). Finally requests stop and joins the worker with a timeout.

```
stop()
```

Signals the stop event. It does not interrupt an inference callback or join a worker by itself.

Functions | nova2026.auditory.audio

`read_audio(path)`

Returns (mono_float_audio, rate). Signed and unsigned PCM are normalized before stereo downmixing. Floating audio is preserved; AudioMixer later enforces absolute amplitude <= 1.

Source: src/nova2026/auditory/audio.py : 10

Functions | nova2026.auditory.data

```
load_kuleuven(path, stimuli, channel_names, reference, upstream_processing)
```

Returns list[AuditoryTrial] from MATLAB v5 and matching WAVs. Sorts candidate filenames independently of attention, then maps the label. Requires channel names/reference/upstream processing; assumes EEG uV and onset-cropped data.

Source: src/nova2026/auditory/data.py : 99

```
load_trial(path)
```

Loads the numeric interchange NPZ with allow_pickle=False and constructs AuditoryTrial. Requires metadata fields produced by save_trial.

Source: src/nova2026/auditory/data.py : 150

```
save_trial(trial, path)
```

Writes eeg, timestamps, audio, labels and JSON metadata. Does not create parent directories.

Source: src/nova2026/auditory/data.py : 171

Functions | nova2026.auditory.evaluation

```
check_split(training, validation, testing=())
```

Returns None or raises ValueError for reused (subject, trial_id) or stimulus groups across partitions. Does not enforce subject-disjoint splits or detect different group names for repeated excerpts.

Source: src/nova2026/auditory/evaluation.py : 8

```
inject_fault(trial, kind, start=8.0, duration=0.25)
```

Deep-copies a trial. dropout inserts NaN; artifact adds 2000 uV; mismatched circularly shifts audio by seven seconds; none leaves the copy unchanged. start is compared directly to trial timestamps.

Source: src/nova2026/auditory/evaluation.py : 25

```
selection_metrics(times, choices, labels)
```

Returns commanded-selection durations, coverage, false selection changes, first-match switch delays and missed switches. choices uses -1 for neutral. Unknown labels are excluded from durations. Final input sample contributes zero duration.

Source: src/nova2026/auditory/evaluation.py : 40

Functions | scripts.auditory.convert

`load_aasd_manifest(path)`

Returns trials from explicitly mapped CNT crops, two matching audio files and attention events. Converts MNE volts to uV, masks switch uncertainty and resolves paths relative to the manifest. Does not discover trigger mappings.

Source: scripts/auditory/convert.py : 14

`main()`

Command entry point. Parses the flags shown below and performs the documented conversion, training, replay or evaluation. Run as python -m from the repository root.

Source: scripts/auditory/convert.py : 74

CLI flag	Definition from source
--kind	choices=['kuleuven', 'aasd']; required=True
--input	required=True
--out	required=True
--stimuli	optional argument
--metadata	help='JSON with channel_names, reference, upstream_processing'

Functions | scripts.auditory.demo

`main()`

Command entry point. Parses the flags shown below and performs the documented conversion, training, replay or evaluation. Run as python -m from the repository root.

Source: scripts/auditory/demo.py : 13

CLI flag	Definition from source
--out	default='output/auditory_demo'

Functions | scripts.auditory.evaluate

`main()`

Command entry point. Parses the flags shown below and performs the documented conversion, training, replay or evaluation. Run as python -m from the repository root.

Source: scripts/auditory/evaluate.py : 14

CLI flag	Definition from source
--trial	required=True
--model	required=True
--out	required=True

Functions | scripts.auditory.replay

```
replay(trial, model, margin=0.03, delay=0.0, audio_offset=0.0, mode='quality')
```

Returns (mono_audio, list_of_estimate_dicts, metrics). Uses a virtual clock and optional delay/offset fault. Modes quality/hysteresis/neutral/oracle. Records processing_failure and renders the remainder. Direct function calls do not enforce held-out trial separation; the CLIs do.

Source: scripts/auditory/replay.py : 20

```
main()
```

Command entry point. Parses the flags shown below and performs the documented conversion, training, replay or evaluation. Run as python -m from the repository root.

Source: scripts/auditory/replay.py : 104

CLI flag	Definition from source
--trial	required=True
--model	required=True
--out	required=True
--mode	choices=['quality', 'hysteresis', 'neutral', 'oracle']; default='quality'
--fault	choices=['none', 'dropout', 'artifact', 'mismatched']; default='none'
--delay	type=float; default=0.0
--audio-offset	type=float; default=0.0
--margin	type=float; default=0.03
--zero-model	action='store_true'

Functions | scripts.auditory.runner

```
stream_config(trial, config, history=5.0, step=1.0)
```

Returns existing StreamConfig with declared reference/channels, uV input, auditory band/rate, no notch, two-second warm-up and buffer_seconds=history+4.

Source: scripts/auditory/runner.py : 19

```
replay_windows(trial, config, history=5.0, step=1.0, chunk_seconds=0.032, max_wait=3.0,
    audio_offset=0.0)
```

Generator of aligned AuditoryWindow objects. Advances a virtual clock; retains bounded pending windows for missing audio. Throws ReplayFailure when processor RuntimeError occurs. Rejects unfinished tails. Nonpositive chunk_seconds is unsupported; validate at the caller.

Source: scripts/auditory/runner.py : 39

```
labels_for_window(trial, window)
```

Returns labels at the preceding source timestamp using searchsorted and clipped indices. Intended for training/evaluation only.

Source: scripts/auditory/runner.py : 109

Functions | scripts.auditory.synthetic

```
synthetic_trial(identity='one', seconds=24)
```

Returns a deterministic synthetic trial with identity-dependent noise/phases. All labels attend A. Suitable for plumbing checks only; no physiological accuracy claim.

Source: scripts/auditory/synthetic.py : 8

Functions | scripts.auditory.train

```
prepare(trials, config, history)
```

Returns valid, nonoverlapping (window, labels) examples selected from one-second-hop causal replay. Materializes examples in memory.

Source: scripts/auditory/train.py : 17

```
train(training, validation, history=5.0, alphas=(10.0, 100.0, 1000.0))
```

Returns (best_model, validation_report). Checks groups, prepares examples, fits each alpha, selects validation accuracy on steady fully labeled windows. Does not refit on validation. Default features use AuditoryConfig().

Source: scripts/auditory/train.py : 28

```
main()
```

Command entry point. Parses the flags shown below and performs the documented conversion, training, replay or evaluation. Run as python -m from the repository root.

Source: scripts/auditory/train.py : 70

CLI flag	Definition from source
--train	nargs='+'; required=True
--validation	nargs='+'; required=True
--model	required=True
--history	type=float; default=5.0

Evaluation boundaries and independent review

Independent software review reproduced four gaps at this snapshot: score can broadcast a one-channel window against a two-channel model; the playback facade stops audio on EEG processing failure; direct replay calls bypass the CLI held-out checks; and duration metrics omit the last rendered block. The accompanying Independent_Review.txt and reproducer record the evidence. Existing auditory (12) and streaming (32) tests passed despite these gaps.

selection_metrics describes commanded selection, not instantaneous acoustic gain during a ramp. Unknown labels are excluded from durations. Switch delay starts at the first known label of the new state and ends at the first matching choice; it does not require stable dwell. The final sampled instant has zero duration. false_selection_changes counts transitions during unchanged known labels, including entry to or exit from neutral.

The synthetic generator always labels A and uses a simple known signal. Existing checks cover software mechanics but are not sufficient evidence of general auditory performance. Use both target directions, real switches, null controls, adverse timing and independent source data in a stronger review.

The separate text brief specifies reproducible commands, adversarial probes, reporting format and hardware boundaries. It asks the reviewing agent to establish findings independently rather than treating this reference or previous test counts as proof of correctness.

This reference documents the implementation snapshot. It is not a declaration that all APIs reject all malformed inputs, that all clock settings are safe, or that the experimental application is ready for a participant study. Test findings should be reported separately without silently rewriting the documented source.

Source fingerprint inventory: NOVA2026_Auditory_API_Source_Manifest.json. This includes all source files used for the signature inventory, so another agent can establish whether the implementation changed after documentation.